

rate-me

Plataforma de feedback de aulas — Documentação técnica

Tech Challenge — Fase 4. Aplicação hospedada em nuvem, com funções serverless para o recebimento de avaliações, o envio de notificações críticas e a geração de relatórios periódicos.

EQUIPE

Igor Costa

Leandro Fita

Thiago Soares

Victor Reis

Rodrigo Ferreira

Repositório: github.com/icgsbr/rate-me · Nuvem: Microsoft Azure · Julho de 2026

Sumário

1. Visão geral e problema
2. Arquitetura da solução
3. Modelo de nuvem adotado
4. Funções serverless
5. Fluxos principais
6. Esquema de banco de dados
7. Estrutura na Azure
8. Segurança e governança de acesso
9. Configuração
10. Deploy e integração contínua
11. Monitoramento
12. Qualidade e testes
13. Atendimento aos requisitos

1. Visão geral e problema

Cursos on-line perdem qualidade em silêncio: o aluno insatisfeito abandona a aula sem que ninguém no time acadêmico fique sabendo a tempo. O rate-me existe para encurtar esse ciclo. O aluno registra uma avaliação da aula com uma descrição e uma nota de 0 a 10; quando essa nota é crítica, o administrador recebe um e-mail em segundos, sem depender de alguém abrir um painel; e, de forma independente do tráfego, um relatório periódico consolida os números do período e chega pronto na caixa de entrada da coordenação.

A solução é composta por quatro componentes que rodam inteiramente na Microsoft Azure: um serviço de autenticação, um serviço de feedback, e duas Azure Functions — uma para relatórios e outra para notificações. A separação não é decorativa: cada componente tem uma responsabilidade única e um ciclo de vida próprio, o que permite atualizar a função de notificação sem tocar no serviço que recebe avaliações.

Tecnologias

Camada	Escolha
Linguagem e runtime	Java 21 (LTS)
Framework	Quarkus 3.15.1
Persistência	PostgreSQL 18 (Azure Database for PostgreSQL Flexible Server), Hibernate ORM com Panache, migrações versionadas com Flyway
Autenticação	JWT RS256 emitido pelo auth-service e validado por JWKS (SmallRye JWT)
Serverless	Azure Functions runtime 4, biblioteca Java 3.1.0, extensão <code>quarkus-azure-functions</code>
Observabilidade	Application Insights (agente Java 3.7.8), Log Analytics, SLF4J + Logback com MDC
Build e testes	Maven multi-módulo, JUnit 5, Mockito, AssertJ, JaCoCo

2. Arquitetura da solução

O desenho separa o que é síncrono e disparado pelo usuário do que é assíncrono e disparado por tempo. As requisições de aluno e administrador chegam a Container Apps com ingress HTTPS; tudo que é periódico — relatório e varredura de logs — vive em Azure Functions com gatilho de tempo, que não consomem recurso enquanto não são acionadas.

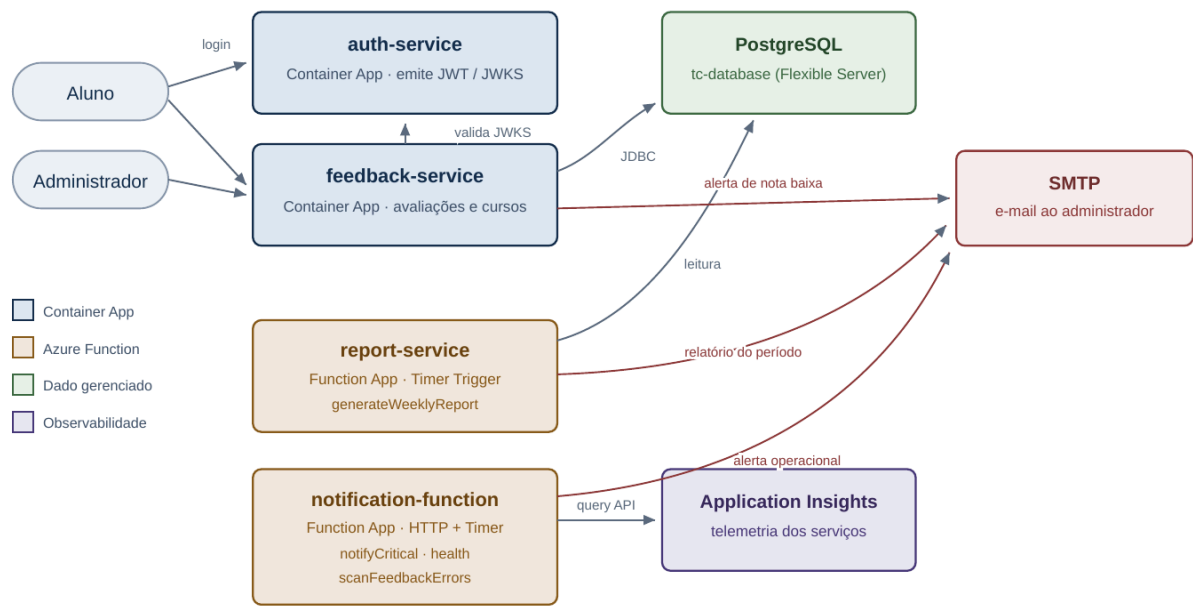


Figura 1 — Componentes da solução e as integrações entre eles.

2.1 Responsabilidade única por componente

Componente	Responsabilidade exclusiva
auth-service	Identidade: cadastro de credencial, emissão do token RS256 e publicação da chave pública. Não conhece avaliações nem cursos.
feedback-service	Aplicação standalone de feedback: cadastro de aluno/administrador, catálogo de cursos, submissão e consulta de avaliações, e a decisão de que uma nota é crítica. É o único que escreve no banco.
report-service	Consolidação: lê as avaliações do período e produz as métricas do relatório. Só lê o banco, nunca altera o esquema nem os dados.
notification-function	Comunicação de incidentes: recebe um evento crítico e o entrega ao administrador, e vigia a telemetria de erro do feedback-service. Não tem acesso ao banco de dados.

2.2 Arquitetura hexagonal

Os três módulos deste repositório seguem o mesmo corte interno. O pacote `domain` contém modelos sem qualquer dependência de framework, incluindo as regras de negócio. O pacote `application` contém os casos de uso e declara portas de entrada e de saída. O pacote `adapter` contém as implementações que tocam o mundo externo, divididas em `input` (recursos REST, gatilhos de função, filtro de segurança) e `output` (JPA, SMTP, cliente REST do auth-service, cliente da API de consulta do Application Insights).

As dependências apontam sempre para dentro. A consequência prática é dupla: trocar o mecanismo de e-mail por outro serviço significa escrever um novo adaptador de `NotificationPort`, sem tocar em um único caso de uso; e a suíte de testes roda sem banco, sem servidor SMTP e sem nuvem, porque cada porta tem um duplê.

```
br.com.fiap.feedback
├── domain                Feedback, Student, Admin, Course, Role, exceções
├── application
│   ├── port.input        SubmitFeedbackUseCase, ListMyFeedbackUseCase, CreateCourseUseCase, ...
│   ├── port.output       FeedbackRepositoryPort, NotificationPort, AuthClientPort, ...
│   └── service            FeedbackService, CourseService, RegisterUserService
└── adapter
    ├── input.web          FeedbackResource, CourseResource, RegistrationResource
    ├── input.security     CurrentUser, MdcRequestFilter
    └── output              persistence/, notification/, auth/
```

3. Modelo de nuvem adotado

A solução roda sobre serviços gerenciados da Azure (PaaS e serverless), cada componente é uma imagem de contêiner que a plataforma executa, escala e reinicia. A escolha do serviço foi feita por natureza da carga.

Carga	Serviço escolhido	Motivo
Requisição/resposta HTTP dos usuários	Azure Container Apps	Precisa de endereço estável, TLS e resposta em milissegundos. O Container Apps entrega ingress HTTPS gerenciado e escala por réplicas, incluindo o retorno a zero quando não há tráfego.
Trabalho disparado por tempo	Azure Functions (Timer Trigger)	Relatório e varredura de logs não têm quem os chame. O agendador do próprio host executa a função e mantém o estado da última execução, eliminando a necessidade de um agendador na aplicação.
Recebimento de evento crítico	Azure Functions (HTTP Trigger)	Endpoint pequeno, sem estado, protegido por chave de função. É a fronteira natural para qualquer produtor de alerta, interno ou externo.
Dados relacionais	PostgreSQL Flexible Server	Banco gerenciado com backup automático, sem administração de instância pela equipe.

As duas Function Apps são publicadas como contêiner próprio. Isso é o que permite manter a mesma imagem, o mesmo runtime Java 21 e o mesmo agente de telemetria em todos os componentes, com o build reproduzível a partir do `Dockerfile` do módulo. Contêiner customizado em Azure Functions exige plano Premium ou Dedicated — o modelo de consumo elástico não aceita imagem própria —, então ambas compartilham um único plano **rate-me-dedicated-plan (B1)**. Reaproveitar um plano para as duas funções mantém o custo do serverless constante ao adicionar a segunda função.

4. Funções serverless

São quatro funções, distribuídas em duas Function Apps. Todas são escritas em Java com Quarkus e injetam os casos de uso da camada de aplicação, o que mantém o gatilho como um adaptador fino.

4.1 generateWeeklyReport — report-service

Gatilho	Timer Trigger, expressão NCRONTAB lida da configuração <code>CRON_SCHEDULE</code> da Function App
Entrada	Nenhuma. A janela é calculada pela própria função: os últimos 7 dias.
Processamento	Consulta as avaliações do período pela porta <code>FeedbackQueryPort</code> e calcula as métricas.
Saída	Relatório registrado pela porta de armazenamento e enviado por e-mail para <code>REPORT_EMAIL</code> .

Conteúdo do relatório, conforme exigido pelo desafio:

- quantidade de avaliações por dia, agrupada por data;
- total de avaliações no período;
- quantidade de avaliações por urgência — as avaliações de nota crítica, com os identificadores dos alunos correspondentes;
- média de avaliações por semana;
- média das notas no período;
- data e intervalo a que o relatório se refere.

4.2 notifyCritical — notification-function

Gatilho	HTTP Trigger, <code>POST /api/notifications/critical</code>
Autorização	Nível <code>FUNCTION</code> : exige a chave no cabeçalho <code>x-functions-key</code> (ou no parâmetro <code>code</code>)
Entrada	JSON com <code>descricao</code> , <code>urgencia</code> e <code>dataEnvio</code>
Saída	<code>200</code> com o evento aceito; <code>400</code> para payload inválido; <code>500</code> quando a entrega falha

```
POST /api/notifications/critical
x-functions-key: <chave da função>
```

```
{
  "descricao": "Database unreachable",
  "urgencia": "CRITICA",
  "dataEnvio": "2026-07-24T00:30:00Z"
}
```

`urgencia` aceita `BAIXA`, `MEDIA`, `ALTA` ou `CRITICA`; `dataEnvio` é opcional e assume o instante do recebimento quando ausente. A validação vive no construtor do registro de domínio `CriticalNotification`, de modo que nenhuma notificação inválida atravessa a camada de aplicação. O e-mail resultante leva a descrição, a urgência e a data de envio.

4.3 scanFeedbackErrors — notification-function

Gatilho	Timer Trigger a cada 5 minutos (<code>LOG_SCAN_SCHEDULE</code>)
----------------	---

Entrada	Estado do agendamento mantido pelo host da função na conta de armazenamento
Processamento	Consulta a API de query do Application Insights do feedback-service por registros de erro na janela
Saída	Quando há erros, um e-mail consolidado para <code>ALERT_ADMIN_EMAIL</code> com até dez ocorrências detalhadas

A janela examinada é `(última execução - atraso, agora - atraso]`, com atraso padrão de 120 segundos para absorver a latência de ingestão da telemetria, e limitada a uma janela máxima para que uma parada longa não gere uma consulta desproporcional. A consulta KQL une `traces` de severidade `ERROR` e `exceptions`, porque uma falha não tratada aparece apenas na segunda tabela.

4.4 health — notification-function

`GET /api/health`, anônimo, responde `{"status":"UP"}`. É a sonda usada para verificar se a Function App está no ar sem precisar de chave — deliberadamente o único endpoint público, e o único que não expõe nem recebe dado algum.

5. Fluxos principais

5.1 Cadastro de usuários e cursos

Nossa aplicação realiza cadastro de usuários e cursos apenas para facilitar o uso e homologação em ambiente de desenvolvimento, o **rate-me** foi pensado para ser uma aplicação standalone focada apenas em cadastro de feedbacks e envio de emails no caso de feedbacks críticos (com nota 1). Em um fluxo de produção, imaginamos que os dados de alunos, administradores e cursos viriam de bases de dados externas, por isso desenvolvemos o **auth-service** focado em autenticação centralizada enquanto o **rate-me** faz apenas autorização das requisições seguindo o padrão **OAuth2 Distributed Security**.

5.2 Submissão de avaliação com alerta de nota baixa

1. O aluno se cadastra em `POST /cadastro`. O feedback-service repassa a credencial ao auth-service e guarda localmente o identificador retornado (`auth_id`), junto do nome e do papel.
2. O aluno autentica diretamente no auth-service e recebe um JWT RS256.
3. Com o token, consulta `GET /courses` e escolhe o curso avaliado.
4. Envia `POST /feedback` com descrição, nota e `courseId`. O serviço valida a faixa da nota, confirma que o curso existe (curso inexistente resulta em `404`) e persiste a avaliação.
5. Se a nota é crítica — igual ou inferior ao limiar definido no domínio —, o caso de uso dispara a notificação ao administrador com descrição, urgência e data de envio, e marca a avaliação como notificada na mesma transação. A marcação impede alerta duplicado e deixa rastro de quando o aviso saiu.

5.3 Geração do relatório

1. O host da Function App aciona `generateWeeklyReport` na expressão configurada.
2. O caso de uso consulta as avaliações dos últimos 7 dias no PostgreSQL, em leitura.
3. As métricas são calculadas em memória e materializadas no registro de domínio `WeeklyReport`.

- 4. O relatório é entregue à porta de armazenamento e enviado por e-mail em formato legível ao destinatário configurado.

5.4 Vigilância dos logs de erro

- 1. A cada cinco minutos, `scanFeedbackErrors` calcula a janela a partir do horário da execução anterior, que o host persiste.
- 2. O adaptador consulta a API de query do Application Insights com uma chave de API dedicada, filtrando pelo nome de papel do feedback-service.
- 3. Sem erros na janela, a execução termina em silêncio — o que é o comportamento esperado na maior parte das execuções.
- 4. Com erros, o serviço monta um resumo com horário, tipo, mensagem e identificador de operação de cada ocorrência e envia um único e-mail de urgência alta, em vez de uma mensagem por erro.

6. Esquema de banco de dados

Uma única base relacional, cujo esquema pertence ao feedback-service. As migrações são aplicadas pelo Flyway na inicialização do serviço, versionadas no próprio repositório; a geração de DDL pelo Hibernate está desligada nos dois serviços, de modo que não existe caminho pelo qual o esquema mude sem uma migração revisada.

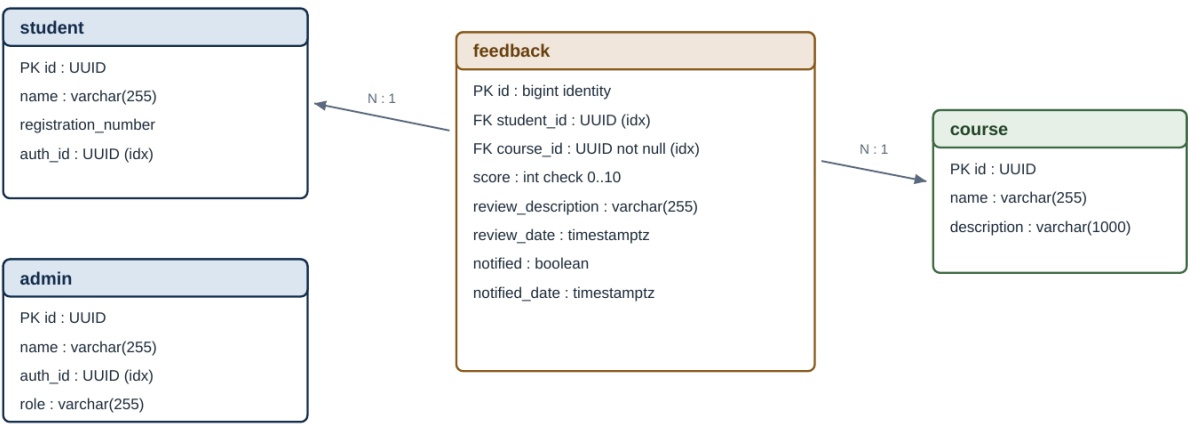


Figura 2 — Modelo de dados. As chaves estrangeiras partem da avaliação para as entidades que ela referencia.

Migração	Conteúdo
V1__init_schema.sql	Cria as quatro tabelas, a restrição de faixa da nota (0 a 10), os índices por <code>auth_id</code> e por aluno, e insere o curso inicial.
V2__course_description_and_required_feedback_course.sql	Acrescenta a descrição ao curso, preenche as avaliações antigas sem curso e torna <code>feedback.course_id</code> obrigatório, com índice próprio.

Os índices existem para os dois acessos frequentes: a resolução do usuário autenticado pelo `auth_id` vindo do token, presente em toda requisição, e a listagem das avaliações de um aluno. As colunas `notified` e `notified_date` registram se e quando o alerta de nota crítica foi enviado, o que torna o envio auditável e idempotente por avaliação.

7. Estrutura na Azure

Todos os recursos vivem no grupo **rg-dev-servless-FIAP**, na assinatura do time. O grupo é a unidade de controle: define o limite de permissão, o inventário e o custo do projeto.

Recurso	Tipo	Papel na solução
auth-service-fiap	Container App	Emissão e publicação de chaves do JWT
feedback-service-fiap	Container App	API de cadastro, cursos e avaliações; ingress externo na porta 8086, escala de 0 a 10 réplicas
managedEnvironment-rgdevse rvlessFI-8ac1	Container Apps Environment	Ambiente compartilhado pelos dois Container Apps, com rede e logs comuns
rate-me-report-service	Function App (contêiner)	Execução agendada do relatório
rate-me-notification-funct ion	Function App (contêiner)	Notificação crítica por HTTP e varredura de logs por tempo
rate-me-dedicated-plan	App Service Plan (B1)	Plano que hospeda as duas Function Apps
tc-database	PostgreSQL Flexible Server	Base relacional: PostgreSQL 18, Standard_B1ms, 32 GB, retenção de backup de 7 dias
ratemeacr	Container Registry	Registro privado das imagens dos quatro componentes
ratemefunctionstorage	Storage Account	Estado do host de Functions, incluindo o histórico de execução dos gatilhos de tempo
Fiap-TC-key-vault	Key Vault	Guarda os segredos de acesso ao banco e os caminhos das chaves de assinatura do JWT
workspacergdevservlessfiap b71d	Log Analytics Workspace	Destino único da telemetria e dos logs de plataforma
*-insights (4 componentes)	Application Insights	Um componente por serviço, apontando para o workspace comum

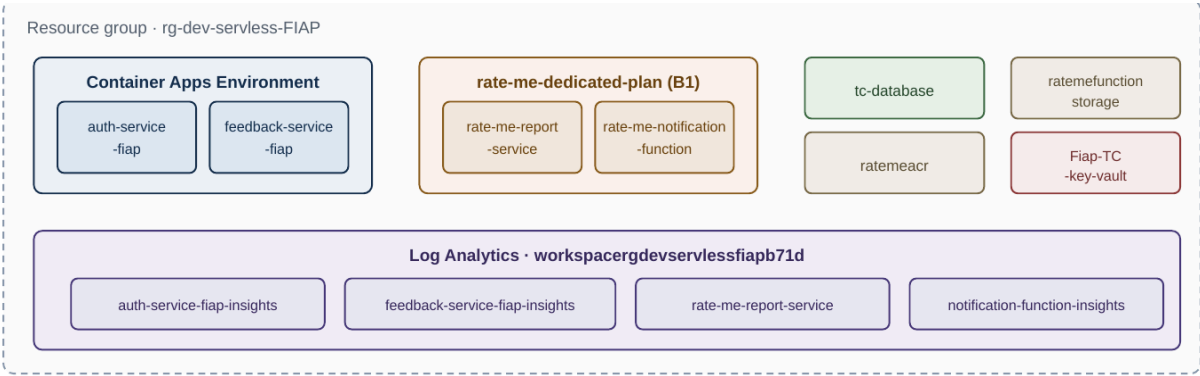


Figura 3 — Organização dos recursos no grupo, com a telemetria de todos os serviços convergindo para um workspace único.

8. Segurança e governança de acesso

8.1 Autenticação e autorização

A identidade é centralizada no auth-service, que assina tokens com RSA (RS256) e publica apenas a chave pública em um endpoint JWKS. O feedback-service nunca vê a chave privada nem a senha do usuário: valida a assinatura de forma distribuída contra o JWKS e confere o emissor esperado. Comprometer um serviço de negócio, portanto, não permite emitir tokens.

O papel do chamador não vem do token: ele é resolvido no próprio feedback-service a partir da reivindicação `authId`, consultando primeiro a tabela de administradores e depois a de alunos. A decisão de autorização fica assim sob controle do serviço dono do dado, e não de um campo que o emissor poderia divergir. Os recursos exigem autenticação por padrão e os casos restritos são explicitados no código com `requireStudent()` e `requireAdmin()`.

Operação	Quem pode
POST /cadastro	Público — é a porta de entrada do cadastro
POST /courses	Somente administrador
GET /courses	Qualquer usuário autenticado
POST /feedback	Somente aluno
GET /feedback	Aluno: apenas as próprias avaliações. Administrador: todas, paginadas
POST /api/notifications/critical	Portador da chave da função
GET /api/health	Anônimo, sem exposição de dados

A restrição de escopo em `GET /feedback` é o ponto que protege dado de aluno: a consulta é sempre parametrizada pelo identificador do próprio chamador, resolvido do token, e não por um parâmetro que o cliente possa alterar.

8.2 Proteção do tráfego e dos segredos

- O ingress dos Container Apps e os endpoints das Function Apps são servidos por HTTPS com certificado gerenciado pela plataforma.
- Nenhuma credencial está no código ou na imagem. Senha do banco, credenciais de SMTP, cadeia de conexão da telemetria e chave da API de consulta são fornecidas como configuração do recurso na Azure, e localmente por variáveis de ambiente a partir do arquivo de exemplo versionado sem valores.
- O Key Vault `Fiap-TC-key-vault` concentra os segredos de acesso ao banco e os caminhos das chaves de assinatura, sob controle de acesso próprio e com histórico de versões.
- As imagens ficam em registro privado (`ratemeacr`); o download é feito com credencial dedicada registrada no recurso, e não com acesso público ao registro.
- O endpoint de notificação crítica usa chave de função, que pode ser rotacionada e revogada por função sem redeploy — a forma de governança de acesso nativa do serviço para uma integração máquina a máquina.

8.3 Governança no plano da assinatura

O isolamento por grupo de recursos delimita o alcance das permissões: os integrantes atuam sobre `rg-dev-servless-FIAP` com papel de colaborador, o que permite operar e publicar os recursos do projeto sem conceder direito de alterar atribuições de papel ou tocar em outros grupos da assinatura. As credenciais usadas pela automação de deploy ficam em segredos do repositório, nunca no histórico de commits.

9. Configuração

Toda propriedade da aplicação tem um valor padrão para desenvolvimento e um ponto de sobrescrita por variável de ambiente. O mesmo artefato de build roda na máquina do desenvolvedor, no docker-compose e na nuvem — o que muda é somente a configuração injetada pelo recurso.

Variável	Onde é definida	Função
DB_URL, DB_USER, DB_PASSWORD	Container App e Function App	Acesso ao PostgreSQL; usado por feedback-service e report-service
AUTH_SERVICE_URL	Container App	Endereço do auth-service para o cadastro
AUTH_JWKS_URL	Container App	Endpoint de chaves públicas para validação do token
JWT_ISSUER	Container App	Emissor esperado; precisa ser idêntico ao configurado no auth-service
MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM	Container App e Function Apps	Servidor SMTP de saída, com TLS obrigatório
ADMIN_EMAIL	Container App e Function App	Destinatário do alerta de nota crítica e da notificação recebida por HTTP
REPORT_EMAIL	Function App do relatório	Destinatário do relatório periódico
ALERT_ADMIN_EMAIL	Function App de notificação	Destinatário do alerta operacional da varredura de logs
CRON_SCHEDULE	Function App do relatório	Expressão NCRONTAB do gatilho de tempo do relatório
LOG_SCAN_SCHEDULE, LOG_SCAN_ENABLED, LOG_SCAN_LAG_SECONDS	Function App de notificação	Periodicidade, chave de desligamento e atraso de ingestão da varredura
FEEDBACK_INSIGHTS_APP_ID, FEEDBACK_INSIGHTS_API_KEY	Function App de notificação	Credenciais da API de consulta do Application Insights monitorado
APPLICATIONINSIGHTS_CONNECTION_STRING	Todos os componentes	Destino da telemetria do agente
JAVA_OPTS	Function App de notificação	Anexa o agente de telemetria e limita o uso de memória da JVM no plano compartilhado

10. Deploy e integração contínua

10.1 Integração contínua

O workflow de CI roda a cada **push** e a cada **pull request** para `main` e `develop`. Ele compila os três módulos, executa as suítes de teste e aplica o portão de cobertura: o build falha se qualquer módulo cair abaixo de 85% de cobertura de linhas. O resumo da execução mostra a tabela de cobertura por módulo, e os relatórios ficam anexados como artefato.

```
mvn -B verify          # compila, testa e verifica a cobertura dos três módulos
```

10.2 Entrega contínua

O deploy dos componentes atualizáveis é automatizado por caminho: alterar o código de uma função publica somente aquela função. Cada workflow autentica na Azure com credencial guardada em segredo do repositório, constrói a imagem a partir do `Dockerfile` do módulo, publica no registro privado com duas etiquetas — o hash do commit, que identifica a versão exata, e `latest`, que a Function App acompanha — e reinicia o recurso para que a nova imagem entre em uso.

Workflow	Disparo	Efeito
ci.yml	push e PR em main/develop	Build, testes e portão de cobertura
report-service-cd.yml	push em main tocando <code>report-service/</code>	Imagem no ACR e reinício de <code>rate-me-report-service</code>
notification-function-cd.yml	push em main tocando <code>notification-function/</code>	Imagem no ACR e reinício de <code>rate-me-notification-function</code>
cd.yml	push em main	Build em matriz das três imagens e publicação no registro público do time

```
docker build -f notification-function/src/main/docker/Dockerfile.jvm \
  -t ratemeacr.azurecr.io/notification-function:latest .
docker push ratemeacr.azurecr.io/notification-function:latest
az functionapp restart --name rate-me-notification-function \
  --resource-group rg-dev-servless-FIAP
```

A construção da imagem é feita em dois estágios a partir da raiz do repositório, porque o build multi-módulo precisa do POM pai: o primeiro estágio compila com Maven sobre JDK 21 e o segundo carrega apenas o artefato final sobre a imagem base do runtime de Azure Functions ou sobre um JRE 21, conforme o componente.

10.3 Ambiente local

O `docker-compose.yml` sobe PostgreSQL e um servidor SMTP de captura, permitindo executar o fluxo completo — incluindo o alerta de nota baixa e o relatório — sem enviar e-mail real e sem depender da nuvem.

```
docker compose up -d postgres mailhog # dependências
cd feedback-service && mvn quarkus:dev # http://localhost:8086
cd report-service && mvn quarkus:dev # http://localhost:8087
```

11. Monitoramento

Cada um dos quatro serviços tem um componente próprio de Application Insights, e todos gravam no mesmo workspace de Log Analytics. A separação por componente evita misturar a telemetria dos serviços; o workspace comum permite correlacionar uma requisição que atravessa mais de um deles.

A instrumentação é feita pelo agente Java do Application Insights, anexado à JVM sem nenhuma linha de código de negócio: requisições, dependências (JDBC, HTTP e SMTP), exceções e métricas de JVM são coletadas automaticamente. O agente é embutido nas imagens durante o build, e o mecanismo de ativação acompanha o tipo de host.

Componente	Forma de ativação	Observação
Container Apps	<code>-javaagent</code> no ponto de entrada da imagem	O contêiner é dono do processo Java
rate-me-notification-function	Configuração <code>JAVA_OPTS</code>	Em contêiner de Function o host é dono do processo, então o agente é anexado por configuração
rate-me-report-service	<code>APPLICATIONINSIGHTS_ENABLE_AGENT</code>	Usa o agente gerenciado pela própria plataforma

A variável `APPLICATIONINSIGHTS_ROLE_NAME` nomeia cada serviço na telemetria, o que torna o mapa de aplicação legível e permite filtrar por serviço nas consultas. No plano da aplicação, um filtro de requisição preenche o contexto de log (MDC) com um identificador de correlação — aceito do cabeçalho `X-Correlation-Id` ou gerado, e devolvido na resposta — além do identificador de autenticação e do papel resolvido. Com isso, todas as linhas de log de uma requisição podem ser reunidas a partir de um único valor, e o contexto é sempre limpo ao final para não vaziar entre requisições que compartilham a mesma linha de execução.

O monitoramento não é apenas passivo. A função `scanFeedbackErrors` consulta a telemetria do `feedback-service` a cada cinco minutos e transforma registros de erro em um e-mail para o administrador, fechando o ciclo entre observar e avisar sem depender de alguém consultar o portal.

12. Qualidade e testes

Os três módulos têm suíte de testes unitários com JUnit 5, Mockito e AssertJ, cobrindo domínio, casos de uso, adaptadores de entrada (recursos REST e gatilhos de função) e adaptadores de saída (persistência, e-mail, cliente de autenticação e cliente de consulta de logs). O portão de cobertura de 85% por módulo é aplicado pelo JaCoCo dentro do `mvn verify`, tanto na máquina do desenvolvedor quanto no CI, o que impede que uma queda de cobertura chegue a `main`.

Nenhum desses testes precisa de banco, servidor SMTP ou nuvem. Isso é consequência direta do desenho hexagonal: cada sistema externo está atrás de uma interface, e o teste fornece um duplê no lugar. Uma consequência prática é que a suíte roda em segundos e o CI não depende de infraestrutura auxiliar. Ficam excluídas da métrica apenas as entidades de persistência e as classes de exceção, que não têm lógica executável própria.

13. Atendimento aos requisitos

Requisito do desafio	Onde está atendido
Ambiente de nuvem configurado e funcionando, com configurações de segurança dos dados de clientes e governança de acesso	Grupo <code>rg-dev-serverless-FIAP</code> com quatro serviços em execução; HTTPS gerenciado, JWT RS256 validado por JWKS, autorização por papel resolvida no serviço dono do dado, chave de função no endpoint de notificação, segredos em configuração de recurso e Key Vault, registro de imagens privado (seções 7 e 8)
Configuração dos componentes de suporte (bancos de dados etc.)	PostgreSQL Flexible Server <code>tc-database</code> com esquema versionado por Flyway; Storage Account para o estado dos gatilhos de tempo; Container Registry para as imagens (seções 6 e 7)
Deploy automatizado dos componentes atualizáveis	Workflows por componente: build da imagem, publicação com etiqueta de commit e reinício do recurso; CI com testes e portão de cobertura antes da entrega (seção 10)
Aplicação monitorada	Application Insights por serviço sobre workspace único, agente Java sem alteração de código, identificador de correlação em log, e alerta operacional automático por e-mail (seção 11)
Notificações automáticas aos administradores para problemas críticos	Alerta de nota crítica disparado na submissão da avaliação, com descrição, urgência e data de envio; função <code>notifyCritical</code> para eventos críticos vindos de qualquer produtor; varredura de erros que avisa sem intervenção humana (seções 4 e 5)
Relatório periódico dos feedbacks, com média de avaliações	<code>generateWeeklyReport</code> , gatilho de tempo sobre a janela de sete dias, com avaliações por dia, avaliações críticas e respectivos alunos, média de avaliações por semana e média das notas (seção 4.1)
Implementação obrigatoriamente serverless, com no mínimo duas funções e responsabilidade única	Quatro funções em duas Function Apps, cada uma com um gatilho e um propósito: relatório, notificação crítica, varredura de logs e sonda de saúde; a separação de responsabilidades também vale entre os serviços (seções 2.1 e 4)
Explicação do modelo de nuvem escolhido e dos componentes envolvidos	Seções 3 e 7: critério de escolha por natureza da carga, motivo do plano dedicado para contêiner em Functions, e inventário dos recursos com o papel de cada um
Qualidade do código com documentação	Arquitetura hexagonal uniforme nos três módulos, código comentado onde a decisão não é óbvia, README do repositório e este documento; cobertura mínima de 85% verificada no CI (seções 2.2 e 12)